

Phase 1 — Arrays, Lists & Basic Collection Problems

1. Find duplicate elements in an array

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4, 1};
        Set<Integer> set = new HashSet<>();
        Set<Integer> duplicates = new HashSet<>();
        for (int num : arr) {
            if (!set.add(num)) {
                duplicates.add(num);
            }
        }
        System.out.println(duplicates);
    }
}
```

2. Find duplicate elements from a list using Stream API

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 2, 4, 1);
        Set<Integer> set = new HashSet<>();
        list.stream()
            .filter(n -> !set.add(n))
            .forEach(System.out::println);
    }
}
```

3. Remove duplicates from a list

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 2, 4, 1);
        Set<Integer> set = new LinkedHashSet<>(list);
        System.out.println(set);
    }
}
```

4. Remove duplicates using Streams

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 2, 4, 1);
        List<Integer> uniqueList = list.stream()
            .distinct()
            .collect(Collectors.toList());
        System.out.println(uniqueList);
    }
}
```

5. Filter even numbers from a list

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1,2,3,4,5,6);
        list.stream()
            .filter(n -> n % 2 == 0)
            .forEach(System.out::println);
    }
}
```

6. Filter odd numbers from a list

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
        List<Integer> oddNumbers = list.stream()
            .filter(n -> n % 2 != 0)
            .collect(Collectors.toList());
        System.out.println("Odd Numbers: " + oddNumbers);
    }
}
```

7. Sort a list in ascending order

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 9, 1, 6);
        List<Integer> sortedList = list.stream()
            .sorted()
            .collect(Collectors.toList());
        System.out.println("Ascending Order: " + sortedList);
    }
}

```

8. Sort a list in descending order

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 9, 1, 6);
        List<Integer> sortedDesc = list.stream()
            .sorted(Comparator.reverseOrder())
            .collect(Collectors.toList());
        System.out.println("Descending Order: " + sortedDesc);
    }
}

```

9. Find max and min numbers using Stream API

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 9, 1, 6);
        Integer max = list.stream()
            .max(Integer::compareTo)
            .get();
        Integer min = list.stream()
            .min(Integer::compareTo)
            .get();
        System.out.println("Max: " + max);
        System.out.println("Min: " + min);
    }
}

```

10. Find the sum of all numbers using reduce()

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 9, 1, 6);
        Integer sum = list.stream()
            .reduce(0, Integer::sum);
        System.out.println("Sum: " + sum);
    }
}
```

11. Find the second highest number from a list

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 9, 1, 6, 9);
        Integer secondHighest = list.stream()
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .findFirst()
            .orElse(null);
        System.out.println("Second Highest: " + secondHighest);
    }
}
```

12. Find top 3 highest numbers from a list

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 2, 9, 1, 6, 8, 7);
        List<Integer> top3 = list.stream()
            .distinct()
            .sorted(Comparator.reverseOrder())
            .limit(3)
            .collect(Collectors.toList());
        System.out.println("Top 3: " + top3);
    }
}
```

13. Find Top N highest salaries using Streams

```
import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    double salary;
    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
    public double getSalary() {
        return salary;
    }
    public String toString() {
        return name + " : " + salary;
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 80000),
            new Employee("C", 70000),
            new Employee("D", 90000),
            new Employee("E", 60000)
        );

        int N = 3;
        List<Employee> topN = employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary).reversed())
            .limit(N)
            .collect(Collectors.toList());
        System.out.println("Top " + N + " Salaries: " + topN);
    }
}
```

14. Find numbers starting with 1 from a list

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(10, 12, 23, 145, 98, 1, 19);
        List<Integer> result = list.stream()
            .map(Integer::valueOf)
            .filter(s -> s.startsWith("1"))
            .map(Integer::valueOf)
    }
}
```

```

        .collect(Collectors.toList());
    System.out.println("Numbers starting with 1: " + result);
}
}

```

15. Merge two sorted arrays

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        int[] arr1 = {1, 3, 5};
        int[] arr2 = {2, 4, 6};
        int[] merged = IntStream
            .concat(Arrays.stream(arr1), Arrays.stream(arr2))
            .sorted()
            .toArray();
        System.out.println("Merged Sorted Array: " + Arrays.toString(merged));
    }
}

```

16. Merge two unsorted arrays into a single sorted array

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        int[] arr1 = {5, 1, 3};
        int[] arr2 = {6, 2, 4};
        int[] merged = IntStream
            .concat(Arrays.stream(arr1), Arrays.stream(arr2))
            .sorted()
            .toArray();
        System.out.println("Merged Sorted Array: " + Arrays.toString(merged));
    }
}

```

17. Find common elements between two lists

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
        List<Integer> list2 = Arrays.asList(4, 5, 6, 7, 8);
        List<Integer> common = list1.stream()
            .filter(list2::contains)

```

```

        .collect(Collectors.toList());
    System.out.println("Common Elements: " + common);
}
}

```

18. Find intersection of two arrays

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3, 4};
        int[] arr2 = {3, 4, 5, 6};
        int[] result = Arrays.stream(arr1)
            .filter(x -> Arrays.stream(arr2).anyMatch(y -> y == x))
            .toArray();
        System.out.println("Intersection: " + Arrays.toString(result));
    }
}

```

19. Skip first 5 elements from a stream

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1,2,3,4,5,6,7,8,9);
        List<Integer> result = list.stream()
            .skip(5)
            .collect(Collectors.toList());
        System.out.println("After Skip: " + result);
    }
}

```

20. Convert List<Integer> to int[]

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 4);
        int[] arr = list.stream()
            .mapToInt(Integer::intValue)
            .toArray();
        System.out.println("Array: " + Arrays.toString(arr));
    }
}

```

```
}
}
```

21. Check whether a list is empty using Optional

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        Optional.ofNullable(list)
            .filter(l -> !l.isEmpty())
            .ifPresentOrElse(
                l -> System.out.println("List is NOT empty"),
                () -> System.out.println("List is empty")
            );
    }
}
```

Phase 2 – String Problems

22. Find the frequency of each character in a string

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        String str = "java";
        Map<Character, Long> freq = str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(
                c -> c,
                Collectors.counting()
            ));
        System.out.println(freq);
    }
}

{a=2, v=1, j=1}
```

23. Count frequency of characters using HashMap

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
```



```

String str = "java";
Map<Character, Integer> map = new HashMap<>();
for (char c : str.toCharArray()) {
    map.put(c, map.getOrDefault(c, 0) + 1);
}
System.out.println(map);
}
}

{a=2, v=1, j=1}

```

24. Find duplicate characters in a string

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "programming";
        Map<Character, Integer> map = new HashMap<>();
        for (char c : str.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        System.out.print("Duplicate characters: ");
        for (Map.Entry<Character, Integer> entry : map.entrySet()) {
            if (entry.getValue() > 1) {
                System.out.print(entry.getKey() + " ");
            }
        }
    }
}

```

Duplicate characters: r g m

25. Find first non-repeated character in String

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "stress";
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char c : str.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        Character result = null;
        for (Map.Entry<Character, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1) {
                result = entry.getKey();
                break;
            }
        }
        System.out.println("First non-repeated character: " + result);
    }
}

```

```
}
```

First non-repeated character: t

26. Find the first non-repeated character in a string

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "stress";
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char c : str.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        Character result = null;
        for (Map.Entry<Character, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1) {
                result = entry.getKey();
                break;
            }
        }
        System.out.println("First Non-Repeated: " + result);
    }
}
```

27. Reverse a String without using inbuilt methods

```
public class Main {
    public static void main(String[] args) {
        String str = "java";
        String reversed = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            reversed += str.charAt(i);
        }
        System.out.println("Reversed String: " + reversed);
    }
}
```

28. Reverse each word using Stream API

```
public class Main {
    public static void main(String[] args) {
        String str = "java";
        StringBuilder sb = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            sb.append(str.charAt(i));
        }
    }
}
```

```

        System.out.println("Reversed String: " + sb.toString());
    }
}

```

29. Reverse words in sentence

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        String sentence = "java stream api";
        String result = Arrays.stream(sentence.split(" "))
            .map(word -> new StringBuilder(word).reverse().toString())
            .collect(Collectors.joining(" "));
        System.out.println(result);
    }
}

```

30. Count the occurrence of each word in a sentence

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        String sentence = "java is easy java is powerful";
        String[] words = sentence.split(" ");
        Map<String, Integer> map = new HashMap<>();
        for (String word : words) {
            map.put(word, map.getOrDefault(word, 0) + 1);
        }
        System.out.println(map);
    }
}

```

{java=2, is=2, easy=1, powerful=1}

Using Stream API

```

Map<String, Long> map = Arrays.stream(sentence.split(" "))
    .collect(Collectors.groupingBy(
        w -> w,
        Collectors.counting()
    ));

```

31. Find the longest string in a list

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("java", "spring", "microservices", "api");
        String longest = list.stream()
            .max(Comparator.comparingInt(String::length))
            .orElse(null);
        System.out.println("Longest: " + longest);
    }
}

```

Longest: microservices

32. Find the shortest string in a list

```

String shortest = list.stream()
    .min(Comparator.comparingInt(String::length))
    .orElse(null);
System.out.println("Shortest: " + shortest);

```

33. Find all palindromes from a list

```

import java.util.*;
import java.util.stream.*;
public class Main {
    static boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }
    public static void main(String[] args) {
        List<String> list = Arrays.asList("madam", "java", "level", "spring", "noon");
        List<String> palindromes = list.stream()
            .filter(Main::isPalindrome)
            .collect(Collectors.toList());
        System.out.println("Palindromes: " + palindromes);
    }
}

```

Palindromes: [madam, level, noon]

34. Find longest substring without repeating characters

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        String s = "abcabcbb";
        Set<Character> set = new HashSet<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
        System.out.println("Longest substring length: " + maxLen);
    }
}

```

Longest substring length: 3

35. Convert list into comma-separated string

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("java", "spring", "boot");
        String result = list.stream()
            .collect(Collectors.joining(","));
        System.out.println(result);
    }
}

```

36. Join list elements using Collectors.joining()

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("java", "spring", "boot");
        String result = list.stream()
            .collect(Collectors.joining());
        System.out.println(result);
    }
}

```

javaspringboot

Phase 3 — Stream API & Functional Programming

37. Convert a list of strings into uppercase

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("java", "spring", "boot");
        List<String> upper = list.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());
        System.out.println(upper);
    }
}
```

[JAVA, SPRING, BOOT]

38. Convert List<String> to Map<String, Integer>

```
Map<String, Integer> map = list.stream()
    .collect(Collectors.toMap(
        s -> s,
        s -> s.length()
    ));
```

39. Convert List to Map using Streams

```
import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    int salary;
    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() { return name; }
    public int getSalary() { return salary; }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> list = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 60000)
        );
    }
}
```

```

);
Map<String, Integer> map = list.stream()
    .collect(Collectors.toMap(
        Employee::getName,
        Employee::getSalary
    ));
System.out.println(map);
}
}

```

40. Flatten a nested list using flatMap()

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<List<Integer>> nested = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4),
            Arrays.asList(5, 6)
        );
        List<Integer> flat = nested.stream()
            .flatMap(List::stream)
            .collect(Collectors.toList());
        System.out.println(flat);
    }
}

```

41. Difference between map() vs flatMap()

42. Sort map based on values

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("A", 3);
        map.put("B", 1);
        map.put("C", 2);
        Map<String, Integer> sorted = map.entrySet().stream()
            .sorted(Map.Entry.comparingByValue())
            .collect(Collectors.toMap(
                Map.Entry::getKey,
                Map.Entry::getValue,
                (e1, e2) -> e1,
                LinkedHashMap::new
            ));
    }
}

```

```

        System.out.println(sorted);
    }
}

```

43. Comparable vs Comparator with examples

```

import java.util.*;
class Employee implements Comparable<Employee> {
    String name;
    int salary;
    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    @Override
    public int compareTo(Employee other) {
        return this.salary - other.salary; // ascending order
    }
    @Override
    public String toString() {
        return name + ":" + salary;
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> list = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 30000),
            new Employee("C", 70000)
        );
        Collections.sort(list);
        System.out.println(list);
    }
}

```

[B:30000, A:50000, C:70000]

```

import java.util.*;
class Employee {
    String name;
    int salary;
    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    @Override
    public String toString() {

```



```

        return name + ":" + salary;
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> list = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 30000),
            new Employee("C", 70000)
        );
        Comparator<Employee> bySalaryDesc =
            (e1, e2) -> e2.salary - e1.salary;
        Collections.sort(list, bySalaryDesc);
        System.out.println(list);
    }
}

```

[C:70000, A:50000, B:30000]

Phase 4 — Employee Stream Problems

44. Sort Employee list using Comparator & Streams

```

class Employee {
    String name;
    int salary;
    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
    public int getSalary() {
        return salary;
    }
    @Override
    public String toString() {
        return name + ":" + salary;
    }
}
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {

```

```

List<Employee> list = Arrays.asList(
    new Employee("A", 50000),
    new Employee("B", 30000),
    new Employee("C", 70000)
);
list.stream()
    .sorted(Comparator.comparing(Employee::getSalary))
    .forEach(System.out::println);
}
}

```

45. Sort employees based on salary using Stream API

```

class Employee {
    String name;
    int salary;
    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
    public int getSalary() {
        return salary;
    }
    @Override
    public String toString() {
        return name + " : " + salary;
    }
}
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 30000),
            new Employee("C", 70000)
        );
        List<Employee> result = employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary))
            .collect(Collectors.toList());
        System.out.println(result);
    }
}

```

46. Find the highest salary employee from a list

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 90000),
            new Employee("C", 70000)
        );
        Employee highest = employees.stream()
            .max(Comparator.comparing(Employee::getSalary))
            .orElse(null);
        System.out.println("Highest Salary Employee: " + highest);
    }
}
```

47. Find total salary of all employees

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 90000),
            new Employee("C", 70000)
        );
        int totalSalary = employees.stream()
            .mapToInt(Employee::getSalary)
            .sum();
        System.out.println("Total Salary: " + totalSalary);
    }
}
```

48. Find Top 3 highest salaries from Employee list

```
import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 90000),
            new Employee("C", 70000),
            new Employee("D", 85000),
        );
    }
}
```

```

        new Employee("E", 60000)
    );
    List<Employee> top3 = employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .limit(3)
        .collect(Collectors.toList());
    System.out.println("Top 3 Highest Salaries: " + top3);
}
}

```

49. Find nth highest salary using Stream API

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 90000),
            new Employee("C", 70000),
            new Employee("D", 85000),
            new Employee("E", 60000)
        );
        int n = 3; // nth highest
        Employee nthHighest = employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary).reversed())
            .distinct()
            .skip(n - 1)
            .findFirst()
            .orElse(null);
        System.out.println(n + "rd Highest Salary Employee: " + nthHighest);
    }
}

```

50. Group Employees by Department using Streams

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", "IT", 50000),
            new Employee("B", "HR", 60000),
            new Employee("C", "IT", 70000),
            new Employee("D", "Finance", 80000),
            new Employee("E", "HR", 55000)
        );
    }
}

```

```

        Map<String, List<Employee>> grouped = employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment));
        System.out.println(grouped);
    }
}

```

51. Group employees by department

```

import java.util.*;
import java.util.stream.*;
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", "IT", 50000),
            new Employee("B", "HR", 60000),
            new Employee("C", "IT", 70000),
            new Employee("D", "Finance", 80000),
            new Employee("E", "HR", 55000)
        );
        Map<String, List<Employee>> grouped = employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment));
        System.out.println(grouped);
    }
}

```

52. Count employees in each department

```

import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    String department;
    Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }
    public String getDepartment() {
        return department;
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", "IT"),
            new Employee("B", "HR"),
            new Employee("C", "IT"),

```

```

        new Employee("D", "Finance"),
        new Employee("E", "HR")
    );
    Map<String, Long> result = employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.counting()
        ));
    System.out.println(result);
}
}

```

53. Find average salary department-wise

```

import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    String department;
    int salary;
    Employee(String name, String department, int salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }
    public String getDepartment() {
        return department;
    }
    public int getSalary() {
        return salary;
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", "IT", 50000),
            new Employee("B", "HR", 60000),
            new Employee("C", "IT", 70000),
            new Employee("D", "Finance", 80000),
            new Employee("E", "HR", 55000)
        );
        Map<String, Double> avgSalaryByDept = employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingInt(Employee::getSalary)
            ));
        System.out.println(avgSalaryByDept);
    }
}

```

```
}
```

54. Partition employees based on salary greater than 50k

```
import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    int salary;
    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    public int getSalary() {
        return salary;
    }
    @Override
    public String toString() {
        return name + ":" + salary;
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 60000),
            new Employee("C", 70000),
            new Employee("D", 45000),
            new Employee("E", 55000)
        );
        Map<Boolean, List<Employee>> partitioned = employees.stream()
            .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
        System.out.println("Salary > 50K: " + partitioned.get(true));
        System.out.println("Salary <= 50K: " + partitioned.get(false));
    }
}
```

55. Find employee names whose salary is greater than average salary

```
import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    int salary;
    Employee(String name, int salary) {
```

```

    this.name = name;
    this.salary = salary;
}
public String getName() {
    return name;
}
public int getSalary() {
    return salary;
}
}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 90000),
            new Employee("C", 70000),
            new Employee("D", 40000),
            new Employee("E", 80000)
        );
        // Step 1: Calculate average salary
        double avgSalary = employees.stream()
            .mapToInt(Employee::getSalary)
            .average()
            .orElse(0);
        // Step 2: Filter employees above average salary
        List<String> result = employees.stream()
            .filter(e -> e.getSalary() > avgSalary)
            .map(Employee::getName)
            .collect(Collectors.toList());
        System.out.println("Average Salary: " + avgSalary);
        System.out.println("Employees above average salary: " + result);
    }
}

```

56. Find department having maximum employees

```

import java.util.*;
import java.util.stream.*;
class Employee {
    String name;
    String department;
    Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }
    public String getDepartment() {
        return department;
    }
}

```



```

}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", "IT"),
            new Employee("B", "HR"),
            new Employee("C", "IT"),
            new Employee("D", "Finance"),
            new Employee("E", "HR"),
            new Employee("F", "IT")
        );
        String maxDept = employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
            ))
            .entrySet()
            .stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
            .orElse(null);
        System.out.println("Department with max employees: " + maxDept);
    }
}

```

Phase 5 — Stack, Queue & Problem Solving

57. Balanced Parentheses problem using Stack

```

import java.util.*;
public class Main {
    public static boolean isBalanced(String str) {
        Map<Character, Character> map = new HashMap<>();
        map.put('(', '(');
        map.put('}', '{');
        map.put(']', '[');
        Stack<Character> stack = new Stack<>();
        for (char ch : str.toCharArray()) {
            if (map.containsKey(ch)) {
                stack.push(ch);
            }
            else if (map.containsValue(ch)) {

```

```

        if (stack.isEmpty() || stack.pop() != map.get(ch)) {
            return false;
        }
    }
}
return stack.isEmpty();
}
public static void main(String[] args) {
    System.out.println(isBalanced("{[]()}"));
}
}

```

58. LRU Cache implementation

```

import java.util.*;
class LRUCache {
    private final int capacity;
    private LinkedHashMap<Integer, Integer> map;
    public LRUCache(int capacity) {
        this.capacity = capacity;
        map = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
                return size() > capacity;
            }
        };
    }
    // Get value
    public int get(int key) {
        return map.getOrDefault(key, -1);
    }
    // Insert key-value
    public void put(int key, int value) {
        map.put(key, value);
    }
    // Display cache
    public void display() {
        System.out.println(map);
    }
    public static void main(String[] args) {
        LRUCache cache = new LRUCache(3);
        cache.put(1, 10);
        cache.put(2, 20);
        cache.put(3, 30);
        cache.display();
        cache.get(1);
        cache.put(4, 40);
        cache.display();
    }
}

```

59. Rate Limiter implementation

Phase 6 – Multithreading & Concurrency

- 60. Producer Consumer problem
 - 61. Deadlock example in Java
 - 62. Thread-safe Singleton implementation
 - 63. CompletableFuture real-time scenario
 - 64. CompletableFuture coding scenario
 - 65. ConcurrentHashMap vs HashMap scenario
 - 66. ConcurrentHashMap vs HashMap coding scenario
-

Phase 7 – Real-Time Backend & System Design Scenarios

- 67. Retry failed REST API calls
 - 68. Retry failed REST API 3 times
 - 69. Kafka duplicate message handling
 - 70. Kafka consumer duplicate message handling
 - 71. Pagination implementation in Spring Boot
 - 72. Pagination implementation using LIMIT & OFFSET
 - 73. Pagination query using LIMIT/OFFSET
-

Phase 8 – Java Design & OOP

- 74. Write Immutable class in Java
-

SQL Interview Questions

SQL Basics

75. Difference between WHERE and HAVING

👉 **WHERE** is used to filter individual rows before grouping, while **HAVING** is used to filter grouped or aggregated data after **GROUP BY**.

76. INNER JOIN vs LEFT JOIN

👉 **INNER JOIN** returns only matching records from both tables, whereas **LEFT JOIN** returns all records from the left table and matching records from the right table, with **NULL** for non-matching rows.

SQL Intermediate

77. Count employees department-wise

```
SELECT department, COUNT(*) AS total_employees FROM employees GROUP BY department;
```

78. Employees having same salary

```
SELECT salary, COUNT(*) AS total_employees FROM employees GROUP BY salary HAVING COUNT(*) > 1;
```

79. Find duplicate rows in table

```
SELECT col1, col2, col3, COUNT(*) AS cnt FROM employees GROUP BY col1, col2, col3 HAVING COUNT(*) > 1;
```

80. Remove duplicate records from table

81. Remove duplicate records from table

```
DELETE FROM Employees WHERE EmployeeID NOT IN ( SELECT MIN(EmployeeID) FROM Employees GROUP BY Name, Department );
```

SQL Advanced

82. Find 2nd highest salary

```
SELECT MAX(salary) AS second_highest_salary FROM employees WHERE salary < ( SELECT  
MAX(salary) FROM employees );
```

83. Find nth highest salary

```
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

84. Department-wise maximum salary query

```
SELECT department, MAX(salary) AS max_salary FROM employees GROUP BY department;
```

85. Department-wise max salary query

86. Pagination using LIMIT & OFFSET

```
SELECT * FROM employees LIMIT page_size OFFSET (page_number - 1) * page_size;
```

Longest substring without repeating characters

First non-repeated character

Duplicate elements using Streams

Group employees by department

Nth highest salary

Comparator vs Comparable

map() vs flatMap()

LRU Cache

Balanced Parentheses

Producer Consumer

CompletableFuture

ConcurrentHashMap vs HashMap

Retry failed REST API

Kafka duplicate handling

Pagination in Spring Boot

Immutable class

Merge arrays

Count frequency using HashMap

Top 3 highest salaries

Convert List to Map using Streams